PEMROGRAMAN BERORIENTASI OBJEK

Laporan Ujian Tengah Semester

KantinPay ITK (E-Wallet Kantin).

Github Repository : https://github.com/frisca210605-hue/KANTIN_PAY.git

Dosen Pengampu :

Himawan Wicaksono, S.ST., M.T.

Disusun Oleh :

FRISCA AULIA WIBISONO 04231035

PROGRAM STUDI TEKNIK ELEKTRO
JURUSAN TEKNIK ELEKTRO, INFORMATIKA, DAN BISNIS
FAKULTAS SAINS DAN TEKNOLOGI INFORMASI
INSTITUT TEKNOLOGI KALIMANTAN
BALIKPAPAN

2025

TAHAP 1: ANALISIS SDLC & MANAJEMEN PROYEK

1. Jelaskan secara ringkas, mengapa sistem pada studi kasus Anda ini lebih aman dan cocok menggunakan paradigma Berorientasi Objek (OOP) dibandingkan paradigma Terstruktur (Prosedural)! Berikan contoh masalah fatal yang mungkin terjadi jika data sensitif pada kasus Anda dibiarkan sebagai variabel global.

   Jawab :

   Sistem e-wallet seperti KantinPay punya risiko tinggi terhadap manipulasi data keuangan kalau tidak dirancang dengan benar. Dibandingkan pendekatan prosedural, paradigma Berorientasi Objek (OOP) cenderung lebih aman karena menerapkan konsep enkapsulasi (data hiding).

   Pada pemrograman prosedural, data sering disimpan sebagai variabel global yang bisa diakses dan diubah oleh fungsi mana pun. Ini membuka celah karena tidak ada pembatasan yang jelas. Sementara itu, dalam OOP, data penting seperti saldo mahasiswa atau PIN disimpan di dalam class dan dilindungi dengan access modifier *private*. Artinya, data tersebut tidak bisa diubah sembarangan, melainkan harus lewat fungsi tertentu (misalnya prosesPembelian()) yang sudah dilengkapi dengan validasi.

   Kalau KantinPay tetap memakai variabel global, beberapa masalah serius bisa muncul. Misalnya, saldo bisa dimanipulasi begitu saja tanpa verifikasi PIN. Selain itu, saldo juga bisa menjadi minus karena tidak ada pengecekan sebelum transaksi dilakukan. Bahkan, transaksi masih bisa terjadi meskipun stok makanan sudah habis, karena sistem tidak membatasi pembelian saat stok nol. Akibatnya, data jadi tidak konsisten dan aturan bisnis dilanggar.

2. Buatlah sebuah repositori GitHub publik. Lakukan inisialisasi Git lokal (git init), buat branch baru bernama fitur-utama, dan berikan screenshot riwayat (log) commit Anda yang rapi (minimal 3 commit) selama mengerjakan proyek ujian ini.

   Jawab :

   Proses manajemen proyek dilakukan dengan menggunakan Git sebagai sistem kendali versi lokal dan GitHub sebagai repositori publik. Langkah-langkah yang dilakukan adalah sebagai berikut:

1) Inisialisasi Repositori dan Branching

Melakukan inisialisasi folder proyek menjadi repositori Git lokal dan membuat branch baru sesuai instruksi dengan nama fitur-utama.

```
git init
git checkout -b fitur-utama
```

2) Identitas Author

Mengatur konfigurasi identitas pengguna agar setiap perubahan tercatat

```
git config --global user.name "Frisca Aulia Wibisono"
```

3) Membuat File Laporan

```
Laporan_UTS_PBO.docx
```

4) Manajemen Commit

Melakukan penambahan file (*staging*) dan perekaman perubahan (*commit*) secara bertahap untuk memastikan riwayat pengerjaan proyek terdokumentasi dengan baik. Berikut adalah contoh perintah yang digunakan untuk mencapai minimal 3 *commit*:

```
# Commit 1
git add  .
git commit -m "docs: analisis paradigma oop vs prosedural kantinpay"

# Commit 2
git add .
git commit -m "docs: analisis paradigma oop vs prosedural kantinpay"

# Commit 3
git add .
git commit -m "docs: melengkapi aturan bisnis dan contoh masalah fatal"
```

5) Menghubungkan ke Repositori Publik (Push)

Menghubungkan repositori lokal ke GitHub dan mengirimkan seluruh perubahan di branch fitur-utama ke server.

```
git remote add origin https://github.com/Friscaaulia/UTS_PBO_KantinPay.git
```

6) Bukti Riwayat Commit (Log)

Berikut adalah tampilan riwayat commit yang telah dilakukan selama pengerjaan proyek:

```
Frisca Aulia@Morfine MINGW64 ~/Documents/UTS_PBO_KANTINPAY (fitur-utama)
$ git log --oneline
f9757dc (HEAD -> fitur-utama) docs: melengkali analisis aturan bisnis saldo dan stok
aba0cc6 feat: merancang strutur class mahasiswa dan menu
c740955 docs: analisis paradigma oop kantinpay
```

a

TAHAP 2: PEMODELAN SISTEM

1. Rancangan Use Case Diagram

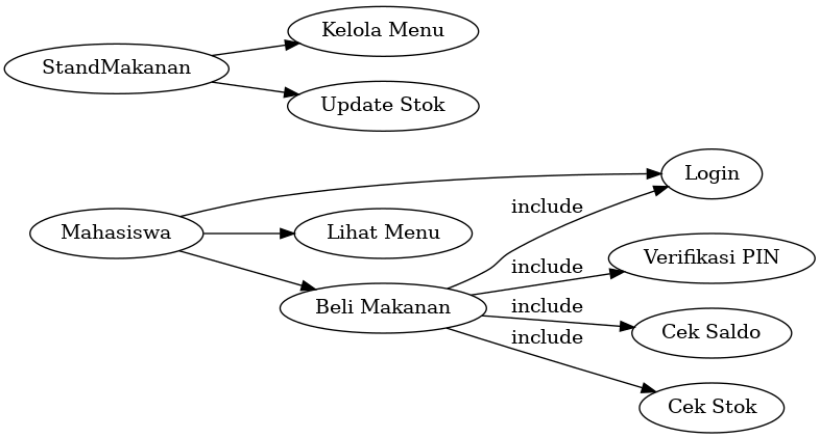
Use Case Diagram digunakan untuk menggambarkan interaksi antara aktor dengan sistem KantinPay. Pada sistem ini terdapat dua aktor utama, yaitu Mahasiswa sebagai pengguna e-wallet dan StandMakanan sebagai penyedia menu.

Mahasiswa dapat melakukan beberapa aktivitas, seperti login ke sistem, melihat daftar menu, serta melakukan pembelian makanan. Dalam proses pembelian, terdapat beberapa tahapan validasi yang harus dipenuhi agar transaksi dapat diproses. Oleh karena itu, use case *Beli Makanan* memiliki relasi <<include>> terhadap beberapa proses lain, yaitu *Login*, *Verifikasi PIN*, *Cek Saldo*, dan *Cek Stok*. Hal ini menunjukkan bahwa sebelum transaksi dilakukan, sistem wajib memastikan bahwa mahasiswa telah terautentikasi, PIN yang dimasukkan benar, saldo mencukupi, dan stok makanan masih tersedia.

Di sisi lain, aktor StandMakanan memiliki tanggung jawab untuk mengelola menu yang tersedia, termasuk menambahkan menu baru serta memperbarui stok makanan. Interaksi ini penting untuk menjaga ketersediaan data menu agar selalu sesuai dengan kondisi nyata di kantin.

Dengan adanya relasi <<include>> pada proses pembelian, sistem menjadi lebih terstruktur dan aman karena setiap transaksi harus melewati tahapan verifikasi yang jelas.

- Gambar Use Case Diagram Kantin Pay ITK



2. Rancangan Class Diagram

Class Diagram pada sistem KantinPay dirancang untuk merepresentasikan struktur objek serta hubungan antar class yang digunakan dalam sistem. Terdapat beberapa class utama yang digunakan, yaitu Mahasiswa, Menu, dan StandMakanan.

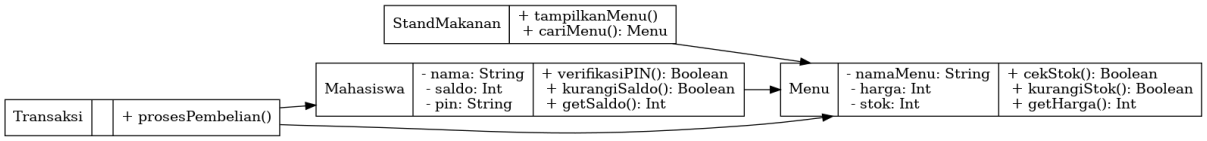
Class Mahasiswa memiliki atribut berupa nama, saldo, dan PIN. Seluruh atribut tersebut bersifat private untuk menjaga keamanan data, khususnya saldo dan PIN yang bersifat sensitif. Class ini menyediakan method public seperti *verifikasiPIN()* untuk memvalidasi input PIN, *kurangiSaldo()* untuk mengurangi saldo saat transaksi, serta *getSaldo()* untuk mengakses informasi saldo secara aman.

Class Menu merepresentasikan makanan atau minuman yang tersedia di kantin. Atribut yang dimiliki meliputi nama menu, harga, dan stok, yang semuanya disimpan sebagai private untuk mencegah manipulasi langsung dari luar class. Method yang disediakan antara lain *cekStok()* untuk memastikan ketersediaan menu, *kurangiStok()* untuk mengurangi jumlah stok setelah transaksi, serta *getHarga()* untuk mengambil nilai harga.

Class StandMakanan berfungsi sebagai pengelola menu. Class ini memiliki atribut berupa daftar menu yang tersedia. Method yang disediakan meliputi *tampilkanMenu()* untuk menampilkan daftar menu kepada mahasiswa dan *cariMenu()* untuk mencari menu tertentu berdasarkan nama.

Hubungan antar class menunjukkan bahwa StandMakanan memiliki relasi agregasi terhadap Menu karena satu stand dapat memiliki banyak menu. Selain itu, terdapat relasi asosiasi antara Mahasiswa dan Menu dalam proses pembelian, di mana mahasiswa berinteraksi dengan menu saat melakukan transaksi.

Dengan penerapan enkapsulasi melalui penggunaan access modifier private pada atribut sensitif, serta penyediaan method public sebagai jalur resmi akses data, rancangan class diagram ini mendukung keamanan sistem sesuai dengan prinsip pemrograman berorientasi objek.



TAHAP 3: IMPLEMENTASI ENKAPSULASI KOTLIN

Pada tahap implementasi, sistem KantinPay dikembangkan dengan menerapkan konsep enkapsulasi untuk menjaga keamanan data sensitif. Data seperti PIN, saldo mahasiswa, dan stok menu tidak dapat diakses maupun diubah secara langsung dari luar class. Hal ini ditunjukkan pada class *Mahasiswa*, di mana atribut PIN dideklarasikan sebagai *private* sehingga tidak bisa diakses dari luar, serta atribut saldo yang menggunakan *private set* agar tidak dapat dimodifikasi secara langsung menggunakan operator penugasan. Dengan pendekatan ini, manipulasi data secara ilegal seperti mengubah saldo tanpa validasi dapat dicegah.

```
class Mahasiswa(
    val idMahasiswa: String,
    val namaMahasiswa: String,
    private val pinRahasia: String,
    saldoAwal: Int
) {
    var saldo: Int = saldoAwal
        private set

    fun prosesPembayaran(totalHarga: Int, inputPin: String): Boolean {
        if (inputPin != pinRahasia) {
            println("❌ GAGAL: PIN yang dimasukkan SALAH!")
            return false
        }
        if (saldo < totalHarga) {
            println("❌ GAGAL: Saldo tidak mencukupi! (Saldo: Rp$saldo, Harga:
Rp$totalHarga)")
            return false
        }
        saldo -= totalHarga
        return true
    }
}
```

Selain itu, pada class *Menu*, atribut stok juga dilindungi menggunakan *private set* sehingga tidak dapat diubah secara langsung. Untuk mengakses dan memodifikasi stok, sistem menyediakan method khusus yang dilengkapi validasi, seperti pengecekan ketersediaan stok dan pengurangan stok setelah transaksi berhasil. Hal ini memastikan bahwa stok tidak akan bernilai negatif dan tetap sesuai dengan kondisi sebenarnya.

```
class Menu(
    val namaMenu: String,
    val harga: Int,
    stokAwal: Int
) {
    var stok: Int = stokAwal
        private set

    fun cekKetersediaan(): Boolean {
        if (stok <= 0) {
            println("❌ GAGAL: Stok menu '$namaMenu' sudah habis!")
            return false
        }
        return true
    }

    fun kurangiStok() {
        if (stok > 0) stok--
    }
}
```

Pengelolaan transaksi dilakukan oleh class *StandMakanan* yang berfungsi sebagai penghubung antar objek. Pada class ini diterapkan logika validasi secara berurutan sesuai aturan bisnis, yaitu pengecekan stok terlebih dahulu, kemudian verifikasi PIN dan saldo. Jika semua kondisi terpenuhi, maka transaksi akan diproses dengan mengurangi saldo mahasiswa dan stok menu. Jika salah satu

kondisi tidak terpenuhi, sistem akan menolak transaksi dan menampilkan pesan kesalahan.

```
class StandMakanan(val namaStand: String) {

    fun prosesTransaksi(mhs: Mahasiswa, menu: Menu, inputPin: String) {
        println("\n--- Memproses Transaksi di $namaStand ---")
                    println("Aktor: ${mhs.namaMahasiswa} mencoba membeli
${menu.namaMenu}")

        if (menu.cekKetersediaan()) {
            if (mhs.prosesPembayaran(menu.harga, inputPin)) {
                menu.kurangiStok()
                println("✅ SUKSES: Transaksi pembayaran berhasil!")
                        println("    -> Saldo ${mhs.namaMahasiswa} terpotong
Rp${menu.harga} (Sisa Saldo: Rp${mhs.saldo}).")
                println("  -> Sisa stok ${menu.namaMenu}: ${menu.stok} porsi.")
            } else {
                println("  -> Transaksi Dibatalkan oleh Sistem KantinPay.")
            }
        } else {
            println("  -> Transaksi Dibatalkan: Makanan tidak tersedia.")
        }
    }
}
```

Untuk membuktikan bahwa sistem bekerja sesuai dengan aturan bisnis, dilakukan simulasi pada fungsi *main()*. Simulasi ini mencakup beberapa kondisi, yaitu transaksi gagal karena stok habis, PIN salah, dan saldo tidak mencukupi, serta transaksi berhasil ketika semua syarat terpenuhi. Hasil simulasi menunjukkan bahwa sistem mampu menolak transaksi yang tidak valid dan hanya memproses transaksi yang sah, sehingga keamanan dan konsistensi data tetap terjaga.

```
fun main() {
```

```
    println("=== SIMULASI SISTEM KANTINPAY ITK ===\n")

    val stand = StandMakanan("Kantin Gedung G")

    // Gagal: Stok habis
    val mhs1 = Mahasiswa("04231038", "Hendra", "123456", 50000)
    val menu1 = Menu("Ayam Geprek", 15000, 0)
    stand.prosesTransaksi(mhs1, menu1, "123456")

    // Gagal: PIN salah
    val mhs2 = Mahasiswa("04231084", "Syamsul", "654321", 50000)
    val menu2 = Menu("Nasi Goreng", 15000, 5)
    stand.prosesTransaksi(mhs2, menu2, "000000")

    // Gagal: Saldo kurang
    val mhs3 = Mahasiswa("04231093", "Zaki", "111111", 10000)
    stand.prosesTransaksi(mhs3, menu2, "111111")

    // Sukses
    val mhs4 = Mahasiswa("04231035", "Frisca", "222333", 100000)
    val menu3 = Menu("Soto Ayam", 12000, 10)
    stand.prosesTransaksi(mhs4, menu3, "222333")
}
```

Secara keseluruhan, implementasi ini telah memenuhi prinsip enkapsulasi dengan baik, di mana data sensitif dilindungi dan hanya dapat dimodifikasi melalui method yang memiliki validasi ketat. Hal ini membuat sistem lebih aman dari manipulasi data dan sesuai dengan aturan bisnis yang telah ditetapkan.

Gambar hasil simulasi :

```
=== SIMULASI SISTEM KANTINPAY ITK ===

>> SIMULASI 1: GAGAL (STOK HABIS)

--- Memproses Transaksi di Kantin Gedung G ---
Aktor: Hendra Firmansyah mencoba membeli Ayam Geprek
❌ GAGAL: Stok menu 'Ayam Geprek' sudah habis!
   -> Transaksi Dibatalkan: Makanan tidak tersedia.
```

```
>> SIMULASI 2: GAGAL (PIN SALAH)

--- Memproses Transaksi di Kantin Gedung G ---
Aktor: Syamsul mencoba membeli Nasi Goreng
❌ GAGAL: PIN yang dimasukkan SALAH!
   -> Transaksi Dibatalkan oleh Sistem KantinPay.
```

```
>> SIMULASI 3: GAGAL (SALDO KURANG)

--- Memproses Transaksi di Kantin Gedung G ---
Aktor: Zaki mencoba membeli Nasi Goreng
❌ GAGAL: Saldo tidak mencukupi! (Saldo: Rp10000, Harga: Rp15000)
   -> Transaksi Dibatalkan oleh Sistem KantinPay.
```

```
>> SIMULASI 4: SUKSES (TRANSAKSI SAH)

--- Memproses Transaksi di Kantin Gedung G ---
Aktor: Frisca Aulia mencoba membeli Soto Ayam
✅ SUKSES: Transaksi pembayaran berhasil!
   -> Saldo Frisca Aulia terpotong Rp12000 (Sisa Saldo: Rp88000).
   -> Sisa stok Soto Ayam saat ini: 9 porsi.
```